

Comprehensive Technical Documentation: Donation Tracker Platform

1. Executive Summary & Platform Motivation

The Donation Tracker is a highly scalable, web-based platform designed to bridge the gap between individuals holding surplus resources and those in need, including verified Non-Governmental Organizations (NGOs). The primary objective of the platform is to eliminate logistical friction in charitable giving by providing a localized, interactive, and transparent ecosystem.

By leveraging modern web technologies, the platform ensures that every transaction is secure, every user is accountable (via Trust Scores), and every NGO is rigorously verified. This document serves as a deep technical dive into the architecture, codebase, algorithms, and libraries that power the Donation Tracker.

2. System Architecture Overview

The platform is built on a monolithic **Client-Server Architecture** utilizing the **Model-View-Controller (MVC)** design pattern, implemented via the Flask Web Framework in Python.

2.1 The MVC Implementation

- **Models (`app/models.py`):** Represents the data structures and database schema. It utilizes SQLAlchemy as an Object-Relational Mapper (ORM) to abstract raw SQL into Python classes.
 - **Views (`app/templates/`):** The presentation layer written in HTML5 and Jinja2 templating language. It dynamically renders server-side data into client-side web pages before sending them over the network.
 - **Controllers (`app/core/`):** Divided into Blueprints (e.g., `auth_routes.py`, `frontend_routes.py`, `profile_routes.py`). These act as the routing mechanism, handling incoming HTTP requests, executing business logic, interacting with the Models, and returning the appropriate Views.
-

3. Database Schema & Data Models

The system relies on a relational database (SQLite for development, adaptable to PostgreSQL/MySQL for production) managed through Flask-SQLAlchemy and Flask-Migrate.

3.1 Core Entities

1. **User Model:** The central entity. It stores `email`, `password_hash`, `role` (user, ngo, admin), and verification statuses. It establishes One-to-Many relationships with Resources, Requests, and PointsTransactions.
 2. **Resource Model:** Represents a physical item donated by a user. It contains geospatial data (`location_lat`, `location_lng`), categorization, and a status enum (Available, Requested, Fulfilled).
 3. **Request Model:** Acts as the transactional bridge between a Receiver and a Resource. It implements a strict State Machine pattern.
 4. **DonationHistory & PointsTransaction:** Ledger tables. They track immutable records of completed exchanges and gamification points awarded to users, ensuring data integrity for Trust Scores.
 5. **UPIDonationClick:** An analytics table tracking intent-to-donate metrics for NGOs, mapped via foreign keys to both the Donor and the NGO.
-

4. Security Algorithms & Authentication Flow

Security is paramount, especially when handling NGO identities and user data. The platform implements a multi-layered security protocol.

4.1 Cryptographic Password Hashing

Raw passwords are never stored. The platform uses the **Bcrypt** algorithm. * **Mechanism:** Bcrypt incorporates a salt to protect against rainbow table attacks and uses an adaptive key setup phase. * **Implementation:** When a user registers, `bcrypt.gensalt()` generates a unique salt, and `bcrypt.hashpw()` computes the hash. Authentication runs the plaintext input through the same algorithm and performs a constant-time string comparison to mitigate timing attacks.

4.2 Time-Based One-Time Password (OTP) Verification

To prevent spam accounts, email verification is strictly enforced during registration. * **Algorithm:** When a user registers, a 6-digit cryptographically secure pseudorandom number is generated using Python's `random` or `secrets` module. * **State Management:** The OTP is stored in an in-memory dictionary (`otp_store`) alongside a UNIX timestamp. * **Validation:** Upon submission, the system calculates `current_time - stored_time`. If the delta exceeds 300 seconds (5 minutes), the OTP is invalidated, forcing a secure timeout.

4.3 Role-Based Access Control (RBAC)

Access to specific endpoints is restricted using custom Python decorators. * **@login_required:** Provided by Flask-Login, it intercepts the HTTP request, checks the session cookie, and redirects unauthenticated users. * **@role_required(*roles):** A custom closure function that wraps routes. It inspects `current_user.role` against the allowed roles tuple. If an unauthorized user (e.g., a standard user trying to access the Admin panel) attempts access, it aborts with an HTTP 403 Forbidden error.

5. Core Business Logic & Algorithms

5.1 The Request Lifecycle (State Machine)

The core algorithm governing resource exchange is a finite state machine applied to the Request model: 1. **State 0: Available:** A resource is listed on the public feed. 2. **State 1: Pending:** User A requests the item. An email is dispatched to the Donor. The Resource remains visible but marked as 'Requested'. 3. **State 2: Accepted:** The Donor reviews the request and accepts it. The Request status updates to Accepted. The Resource is hidden from the public feed. 4. **State 3: Fulfilled:** The physical exchange occurs. The Donor marks it complete. The system triggers the Gamification Algorithm. 5. **State 4: Rejected / Unfulfilled:** If the Donor rejects the request, the Request is marked Rejected, and the Resource reverts to **State 0 (Available)** for other users.

5.2 Gamification & Trust Score Algorithm

To encourage good behavior, the system assigns points. * When a resource is successfully Fulfilled, the system instantiates a `PointsTransaction`. * **Formula:** `User.points_balance += 10, User.trust_score += 1`. * **Badge Calculation:** A dynamic algorithm evaluates the `trust_score`. E.g., `score > 10` = Bronze, `score > 50` = Silver, `score > 100` = Gold. This badge is dynamically rendered on their public profile.

5.3 Image Processing Algorithm

To save bandwidth and storage, user uploads (ID documents, profile photos, gallery images) are processed server-side before storage. * **Library:** Pillow (PIL). * **Process:** The file is opened into memory. The `thumbnail()` method applies a high-quality downsampling filter (LANCZOS) to proportionally reduce dimensions (e.g., max 800x800). It is then saved with `optimize=True` and `quality=85`, reducing file sizes by up to 80% while maintaining visual fidelity.

6. Frontend Engineering & User Experience

6.1 UI/UX Paradigm: Glassmorphism

The frontend heavily utilizes **Glassmorphism**, a modern UI trend characterized by translucent, frosted-glass effects. * **CSS Implementation:** Achieved using `backdrop-filter: blur(12px)` combined with a semi-transparent `rgba(255, 255, 255, 0.1)` background and subtle white borders. This creates depth and hierarchy on the Z-axis against vibrant CSS gradient backgrounds.

6.2 Responsive Grid Systems

- **Bootstrap 5:** The core CSS framework provides a 12-column flexbox grid. The application utilizes breakpoint classes (`col-md-6`, `col-lg-4`) to ensure the layout seamlessly reflows from desktop monitors to mobile devices.
- **Masonry Layout:** The resource feed uses a CSS column-count implementation (or JavaScript masonry) to pack items of varying heights tightly together, maximizing screen real estate.

6.3 Asynchronous DOM Updates

Features like the "Donate via UPI" button utilize AJAX (fetch API). When clicked, an asynchronous POST request is sent to `/api/track-upi-click`. The server records the interaction and returns a 200 OK status without requiring a full page reload, resulting in a frictionless user experience.

7. API Design & Routing Logic

The application relies on a strictly defined set of REST-like endpoints categorized via Flask Blueprints. This ensures modularity and separation of concerns.

7.1 Authentication Endpoints (`auth_routes.py`)

- GET `/auth/register` - Renders the registration form.
- POST `/auth/register` - Captures email, password, role, and `id_document`. Generates the OTP and caches the temporary user data in a secure server-side session.
- POST `/auth/verify-email-otp` - Validates the user's OTP input. If successful, permanently saves the uploaded document to `static/uploads/documents`, hashes the password, and creates the User database record.
- POST `/auth/login` - Authenticates user credentials using Bcrypt and establishes the session cookie via Flask-Login.

7.2 Core Application Endpoints (`frontend_routes.py` & `request_routes.py`)

- GET `/` - Queries the Resource table. Applies filtering based on URL parameters (e.g., `?category=Food`) and paginates the results to prevent overloading the DOM.
- POST `/request/<resource_id>` - Requires authentication. Validates that the resource isn't already claimed, creates a new Request record tied to the user and resource, and invokes the asynchronous Email Service.

7.3 Profile & Administration Endpoints (`profile_routes.py` & `admin_routes.py`)

- GET `/profile` - Aggregates data across three complex queries: Incoming requests (where the user is the donor), outgoing requests (where the user is the receiver), and total point transactions.
 - POST `/admin/approve_user/<user_id>` - Requires Admin role. Flips the user's `verification_status` to approved and sends an acceptance email to the user.
-

8. Error Handling, Logging, and Data Integrity

Building a resilient application requires anticipating failure points and gracefully degrading rather than crashing.

8.1 Defensive Programming

- **Foreign Key Constraints:** At the database level, operations like deleting a user automatically cascade to their requests or resources (or set them to NULL depending on the business rule), preventing orphaned data.
- **NoneType Safeguards:** Jinja2 templates are strictly guarded. E.g., `{{ request.resource.title if request.resource else 'Deleted Item' }}` ensures that if a resource was hard-deleted by an admin, the profile page still renders without throwing a 500 Internal Server Error.

8.2 Client-Side Feedback (Flash Messaging)

- The platform utilizes Flask's `flash()` mechanism. When an error occurs (e.g., "Invalid OTP" or "File too large"), the session stores a message category (danger, success, warning).
 - Upon the next page render, a global template component intercepts these messages and displays them as non-intrusive Bootstrap Toast notifications or Alert banners.
-

9. Testing & Environment Strategies

To maintain code quality and ensure reproducible deployments, strict environmental separation is enforced.

9.1 Environment Variables

Sensitive data is completely abstracted from the codebase using the `python-dotenv` library. A `.env` file (excluded from version control via `.gitignore`) holds: * `SECRET_KEY` (Used for session signing) * `MAIL_USERNAME` / `MAIL_PASSWORD` (SMTP credentials) * `DATABASE_URL` (Connection string)

9.2 Database Seeding

To assist developers and QA testers, the `run.py` entry point includes a `seed_db()` function. When initialized on a fresh database, this algorithm automatically generates a predefined Admin, a verified NGO, and standard community members with hashed passwords, allowing immediate interaction with the platform without manually navigating the registration funnel.

10. Future Scalability & Platform Roadmap

While the current architecture handles moderate traffic gracefully, the platform is designed with future enterprise-scale growth in mind.

10.1 Database Migration to PostgreSQL

The abstraction provided by SQLAlchemy means the underlying database can be swapped with zero changes to the business logic. Moving from SQLite to PostgreSQL will unlock concurrent write capabilities, JSONB data types, and horizontal scaling via read-replicas.

10.2 Asynchronous Task Queues (Celery + Redis)

Currently, sending emails (via SMTP) blocks the main web request thread for ~1.5 seconds. The immediate roadmap includes introducing **Celery** (a distributed task queue) backed by a **Redis** in-memory datastore. * When a user requests an item, the email dispatch logic will be handed off to Celery. * The web server will immediately return a response to the user, while a background Celery worker process handles the SMTP negotiation asynchronously.

10.3 OTP and Session Management Upgrade

As the user base grows, the current in-memory dictionary `otp_store` will become a bottleneck across distributed servers. The roadmap dictates migrating OTP storage to **Redis**. This allows multiple Gunicorn worker nodes to share access to OTP states with built-in Time-To-Live (TTL) auto-expiration, entirely replacing the `Python time.time()` calculation.

10.4 Containerization (Docker)

To eliminate "it works on my machine" discrepancies, the platform will be wrapped in a Dockerfile. A `docker-compose.yml` file will orchestrate the Flask Web Server, the PostgreSQL database, the Redis cache, and the Celery workers into isolated, deployable containers.

11. Geospatial Mapping Integration (Leaflet.js)

To facilitate local resource exchanges, the platform integrates robust geospatial mapping features.

11.1 Client-Side Map Rendering

- **Library Selection:** The platform utilizes **Leaflet.js**, a highly lightweight open-source JavaScript library for mobile-friendly interactive maps.
- **Tile Providers:** OpenStreetMap (OSM) tile servers provide the foundational cartography layer. These tiles are cached and rendered asynchronously to prevent blocking the main DOM thread.

11.2 Geocoding and Reverse Geocoding

When a donor lists a new item, they are not required to manually type their coordinates. *

Algorithm: The platform interfaces with the **Nominatim API** (OSM's search engine). When a user types a plain-text address (e.g., "123 Main St"), JavaScript intercepts the input and sends an AJAX request to Nominatim. * **Data Transformation:** Nominatim returns a JSON object containing the exact Latitude and Longitude. The frontend automatically plots a draggable marker on the map using these coordinates. * **Database Storage:** The coordinates are then passed to the Flask backend via hidden form fields and stored securely as `Float` data types in the `Resource` table (`location_lat` and `location_lng`).

12. User Flow & Sequence Diagrams

Understanding the platform requires mapping the exact sequence of events for core operations.

12.1 The Registration Sequence

1. **Client:** Submits Email, Password, Role, and ID Document.
2. **Server:** Validates data. Saves ID Document securely to disk. Generates a 6-digit OTP.
3. **Server:** Sends OTP via SMTP to the provided email.
4. **Server:** Stores temporary user data in session cookie. Redirects Client to OTP verification view.
5. **Client:** Enters OTP.
6. **Server:** Validates OTP against in-memory store. Hashes password. Commits new User to database with status pending.
7. **Client:** Redirected to login page with success notification.

12.2 The Donation Exchange Sequence

1. **Receiver:** Clicks "Request" on a Resource on the Home Feed.
2. **Server:** Validates Receiver is authenticated. Creates Request record. Sends email to Donor.
3. **Donor:** Logs in, navigates to Profile. Sees pending incoming request. Clicks "Accept".
4. **Server:** Updates Request status to 'Accepted'. Hides Resource from public feed. Sends acceptance email to Receiver.
5. **Receiver & Donor:** Communicate externally to exchange the physical item.

6. **Donor:** Navigates back to Profile, clicks "Fulfill".
 7. **Server:** Awards Trust Score points to Donor. Finalizes database records.
-

13. Comprehensive UI Component Breakdown

The User Interface is constructed from meticulously designed, reusable components.

13.1 Navigation Hierarchy

The overarching navigation bar is persistent across all views. * **Authenticated State:** Displays links to the Home Feed, Add Resource, Profile, and Logout. * **Admin State:** Dynamically renders a red "Admin Panel" link if `current_user.role == 'admin'`. * **NGO State:** Exposes additional links for uploading NGO Gallery items and editing the NGO Mission Statement.

13.2 Profile Dashboard Architecture

The Profile view (`profile.html`) is the most complex UI structure in the application, split into distinct functional quadrants: 1. **User Identity Card (Top Left):** Displays the compressed Avatar image, User Name, Role Badge, and a dynamic Trust Score progress bar utilizing CSS linear gradients. 2. **Action Centers (Top Right):** Dynamic buttons allowing NGOs to upload gallery images or users to edit profile pictures. 3. **Incoming Requests Table (Bottom Left):** A paginated, responsive Bootstrap table iterating over `incoming_requests`. It includes conditional Jinja logic to render green "Accept" buttons or red "Reject" buttons based on the `Request.status`. 4. **Outgoing Requests Timeline (Bottom Right):** A visual timeline showing the status of items the user has requested, providing instant visual feedback via colored badges (Pending=Yellow, Accepted=Blue, Fulfilled=Green).

14. Maintenance & Administration Manual

For system administrators responsible for the platform's uptime and integrity, specific procedures are required.

14.1 Log Analysis and Troubleshooting

- The application currently logs to `stdout` via Gunicorn.
- When encountering a 500 Internal Server Error, administrators should search the logs for `SQLAlchemyError` or `TemplateNotFound`.
- The most common error in development is a mismatch between the database schema and Python models. This is resolved by running `flask db migrate` followed by `flask db upgrade` in the terminal.

14.2 Manual Database Interventions

While the Admin Dashboard provides UI tools for banning users and approving NGOs, critical database corruption (such as invalid Enums) requires CLI intervention: * Using the command `sqlite3 app.db`, administrators can execute raw SQL queries. * Example: If an orphaned request is crashing a profile page, an admin can run `DELETE FROM requests WHERE resource_id IS NULL`; to sanitize the database.

15. Continuous Integration / Continuous Deployment (CI/CD)

To transition this project from development to an enterprise-grade production environment, a formal CI/CD pipeline is designed.

15.1 The GitHub Actions Pipeline

- **Push Event Trigger:** Every commit pushed to the main branch triggers the pipeline.

- **Stage 1: Linting:** The pipeline installs `flake8` and scans the Python codebase for syntax errors, PEP8 style violations, and cyclomatic complexity warnings.
 - **Stage 2: Unit Testing:** The pipeline executes `pytest`. It spins up a temporary in-memory SQLite database, runs the mock authentication and request flows, and generates a test coverage report.
 - **Stage 3: Deployment:** If Stages 1 and 2 pass with exit code 0, the pipeline sends a webhook to the production server (e.g., Render.com or AWS Elastic Beanstalk) to automatically pull the latest code, install new dependencies from `requirements.txt`, restart Gunicorn, and serve the new application to users with zero downtime.
-

16. Data Privacy & GDPR Considerations

In a platform handling user emails, phone numbers, and location data, stringent privacy mechanisms are enforced.

16.1 Minimal Data Collection

The platform operates on a principle of data minimization. Users are only required to provide an email and password. Phone numbers are strictly optional.

16.2 Location Data Masking

While donors provide specific coordinates for resources, the platform is designed so that exact coordinates can be slightly obfuscated on the public map (e.g., snapping to the nearest 100 meters) to protect the physical privacy of the donor until a request is officially accepted.

16.3 Right to Erasure

The database schema utilizes `cascade="all, delete-orphan"` on critical relationships. When a user account is deleted by an administrator, all associated Resources, Requests, and Messages are automatically and permanently purged from the database, satisfying standard data protection regulations.

17. Third-Party Libraries & Dependencies

The `requirements.txt` file outlines the critical external packages powering the platform:

1. **Flask:** The micro web framework that acts as the backbone of the application.
 2. **Flask-SQLAlchemy:** An extension that adds support for SQLAlchemy.
 3. **Flask-Login:** Manages user sessions and secure cookies.
 4. **Flask-Mail:** Wraps Python's `smtpLib` for TLS/SSL encrypted emails.
 5. **Bcrypt:** Cryptographic library explicitly designed for password hashing.
 6. **Pillow:** The Python Imaging Library (fork) for image compression.
 7. **Werkzeug:** A WSGI web application library handling `secure_filename()`.
 8. **Gunicorn:** A Python WSGI HTTP Server for UNIX, capable of handling concurrent requests by spawning multiple worker processes.
-

18. Conclusion

The Donation Tracker is a technically robust, secure, and modern application. By combining strict backend verification algorithms with a highly responsive, aesthetically pleasing frontend, and maintaining a clear architectural pathway for enterprise scaling, the platform is perfectly positioned to handle real-world charitable logistics effectively.